Sonar QnA

1. What does the SonarQube CLI do?

The SonarQube CLI is a command-line tool (the “sonar” command) that developers run in their own terminal. It talks directly to your SonarQube Cloud or SonarQube Server account and lets you: • Scan a file, a git commit, or your uncommitted changes for hardcoded secrets and vulnerabilities, right from your terminal. • Get fast, local feedback on code you're about to commit even before it ever reaches CI. • Look up your projects and open issues as JSON, CSV, or a table. This is useful for scripts and dashboards. • Install SonarQube support into AI coding agents like Claude Code or GitHub Copilot CLI with one command. In short: it's a tool a person (or a script) runs by hand, one command at a time.

2. What does the SonarQube MCP Server do?

The MCP Server isn't something you run commands in infact it's a bridge. It exposes SonarQube's analysis, issues, quality gates, security hotspots, and coverage data as a set of “tools” that an AI agent (Claude Code, Cursor, GitHub Copilot, etc.) can call on its own, as part of a conversation or an autonomous coding task. You set it up once, and after that the AI agent decides when to use it and you don't type MCP commands yourself.

3. How would you explain the difference between them to someone who's never used either?

Think of the CLI as a steering wheel and the MCP Server as an engine part that other machines plug into. • The CLI is something you operate directly - you type a command, you get a result. • The MCP Server isn't operated by a person at all - it's a connector that lets an AI agent operate SonarQube on your behalf, deciding for itself when to check for issues or verify a quality gate. Put simply: use the CLI when you (a human) want to run a check yourself. Use the MCP Server when you want your AI coding assistant to be able to check for you, automatically, while it works.

4. What are some good things about these docs in general, and about page [4] specifically?

- The CLI page pre-empts a real confusion point There are actually two different tools with almost the same name: SonarQube CLI and SonarScanner CLI. Someone new could easily grab the wrong one, use it, and get confused when it doesn't do what they expected — that's exactly the kind of mix-up that generates a support ticket.The CLI page notices this risk and deals with it head-on. It has a section that says, plainly, "here's how SonarQube CLI is different from SonarScanner CLI." So instead of waiting for a reader to get confused and go looking for help, the docs answer the question before anyone even has to ask it.
- The page 4 doesn't stop at "installed." The "Check the status" section tells the reader exactly where to confirm the extension is actually healthy, which closes the loop instead of leaving success ambiguous.

5. How would you improve these docs in general, and page [4] specifically?

The overall look and feel of the docs does not look professional and can be improved in terms of format, language, structure. I have seen some emojis or symbols used on pages which does not suit the documentation set. The layout can be more organised. The TOC can be improved by using better page navtitles and structure. Eg: First tab is What is Sonarqube and next is About Sonar. Does not feel right.